



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального
государственного бюджетного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _ ИУК «Информатика и управление»

КАФЕДРА ИУК5 «Системы обработки информации»

О Т Ч Е Т

УЧЕБНАЯ ПРАКТИКА

«Учебно-технологический практикум»

Лабораторная работа № 2

«Реализация основных алгоритмических конструкций»

Студент гр. ИУК5-13Б _____ (Амбарцумян Александр Арташесович)

(подпись) (Ф.И.О.)

Руководитель _____ (Кондратьева Светлана Дмитриевна)

(подпись) (Ф.И.О.)

Оценка руководителя _____ баллов _____

(дата)

Оценка защиты _____ баллов _____

(дата)

Оценка практики _____ баллов _____

(оценка по пятибалльной шкале)

Комиссия:

(подпись) (Ф.И.О.)

(подпись) (Ф.И.О.)

(подпись) (Ф.И.О.)

Калуга, 2026

Цель:

Сформировать практические навыки по основам построения базовых алгоритмов и их реализации на языке C++.

Задачи:

1. Овладеть навыками построения базовых алгоритмов.
2. Овладеть навыками выбора оптимальных алгоритмов программирования.
3. Овладеть навыками описания основных этапов построения алгоритмов.
4. Овладеть навыками реализации основных алгоритмических конструкций: линейной, разветвляющейся и циклической.
5. Изобразить разработанные алгоритмы в виде блок-схем в соответствии с ГОСТ 19.701-90 и оформить отчет.

Вариант 1**№ 1**

Даны x , y . Получить:

$$z = \frac{|x| - |y|}{1 + |xy|}$$

Ход решения:

- 1) Ввести с клавиатуры числа x и y . Переменные объявлены вещественными (тип `double`), так как числа могут быть дробными.
 - 2) Вычислить значение выражения по формуле: $z = (|x| - |y|) / (1 + |x \cdot y|)$. Модуль числа вычисляется функцией `fabs()` из библиотеки `cmath`.
 - 3) Вывести полученное значение на экран.
- Знаменатель $1 + |x \cdot y|$ всегда не меньше единицы, поэтому деления на ноль не возникает при любых значениях x и y . Алгоритм является линейным.

Блок-схема алгоритма:

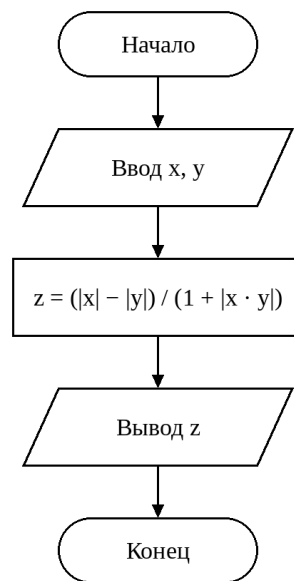


Рисунок 1 – Блок-схема алгоритма задачи № 1

Код программы:

```
1  #include <locale>
2  #include <cmath>
3  #include <iostream>
4
5  int main() {
6      setlocale(LC_ALL, "Russian");
7      double x, y;
8      std::cout << "Введите x: ";
9      std::cin >> x;
10     std::cout << "Введите y: ";
11     std::cin >> y;
12
13     double z = (fabs(x) - fabs(y)) / (1 + fabs(x * y));
14
15     std::cout << "Результат: " << z << "\n";
16     return 0;
17 }
```

Рисунок 2 – Код программы задачи № 1

Результат работы программы:

```
Введите x: 3
Введите y: 2
Результат: 0.142857
```

Рисунок 3 – Результат работы программы задачи № 1 (x = 3, y = 2)

```
Введите x: -4
Введите y: 1.5
Результат: 0.357143
```

Рисунок 4 – Результат работы программы задачи № 1 ($x = -4$, $y = 1,5$)

```
Введите x: 0
Введите y: 5
Результат: -5
```

Рисунок 5 – Результат работы программы задачи № 1 ($x = 0$, $y = 5$)

Проверка: при $x = 3$, $y = 2$ получаем $z = (3 - 2) / (1 + 6) = 1 / 7 \approx 0,142857$; при $x = -4$, $y = 1,5$ получаем $z = (4 - 1,5) / (1 + 6) = 2,5 / 7 \approx 0,357143$; при $x = 0$, $y = 5$ получаем $z = (0 - 5) / 1 = -5$. Результаты совпадают с выводом программы.

№ 2

Даны 2 числа. Получить среднее арифметическое кубов этих чисел и среднее геометрическое модулей этих чисел.

Ход решения:

- 1) Ввести с клавиатуры два вещественных числа a и b .
- 2) Вычислить среднее арифметическое кубов: $\text{cubeMean} = (a^3 + b^3) / 2$.
- 3) Вычислить среднее геометрическое модулей: $\text{geomMean} = \sqrt{(|a| \cdot |b|)}$. Модули берутся для того, чтобы подкоренное выражение было неотрицательным; корень вычисляется функцией `sqrt()`.
- 4) Вывести оба значения на экран.

Блок-схема алгоритма:

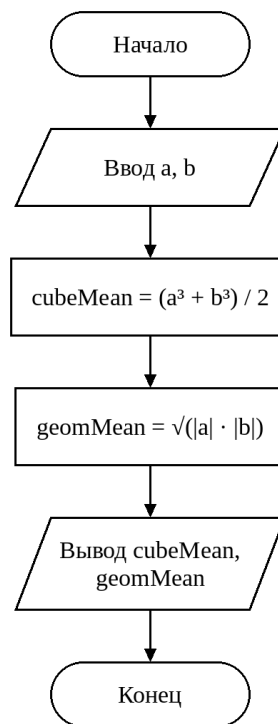


Рисунок 6 – Блок-схема алгоритма задачи № 2

Код программы:

```
1  #include <locale>
2  #include <cmath>
3  #include <iostream>
4
5  int main() {
6      setlocale(LC_ALL, "Russian");
7      double a, b;
8      std::cout << "Введите два числа: ";
9      std::cin >> a >> b;
10
11     double cubeMean = (a * a * a + b * b * b) / 2;
12     double geomMean = sqrt(fabs(a) * fabs(b));
13
14     std::cout << "Среднее арифметическое кубов: " << cubeMean << "\n";
15     std::cout << "Среднее геометрическое модулей: " << geomMean << "\n";
16     return 0;
17 }
```

Рисунок 7 – Код программы задачи № 2

Результат работы программы:

```
Введите два числа: 2 4
Среднее арифметическое кубов: 36
Среднее геометрическое модулей: 2.82843
```

Рисунок 8 – Результат работы программы задачи № 2 ($a = 2$, $b = 4$)

```
Введите два числа: -3 3
Среднее арифметическое кубов: 0
Среднее геометрическое модулей: 3
```

Рисунок 9 – Результат работы программы задачи № 2 ($a = -3$, $b = 3$)

Проверка: $(2^3 + 4^3) / 2 = (8 + 64) / 2 = 36$, $\sqrt{(2 \cdot 4)} = \sqrt{8} \approx 2,82843$; $((-3)^3 + 3^3) / 2 = (-27 + 27) / 2 = 0$, $\sqrt{(3 \cdot 3)} = 3$.

№ 3

Определить, попадает ли заданная точка внутрь заданной области:

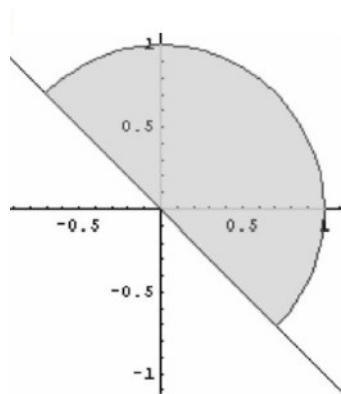


Рисунок 10 – Заданная область

Ход решения:

1) Описать область аналитически. Область на рисунке 10 ограничена окружностью радиуса 1 с центром в начале координат и прямой $y = -x$. Точка принадлежит области, если она лежит внутри круга ($x^2 + y^2 \leq 1$) и не ниже прямой ($y \geq -x$). Точки границы считаются принадлежащими области.

2) Ввести с клавиатуры координаты точки x и y .

3) Проверить составное условие $x^2 + y^2 \leq 1$ и $y \geq -x$. Условия объединяются логической операцией $\&\&$ (и), так как должны выполняться одновременно.

4) Если условие истинно, вывести сообщение «Точка попадает в область», иначе — «Точка не попадает в область».

Блок-схема алгоритма:

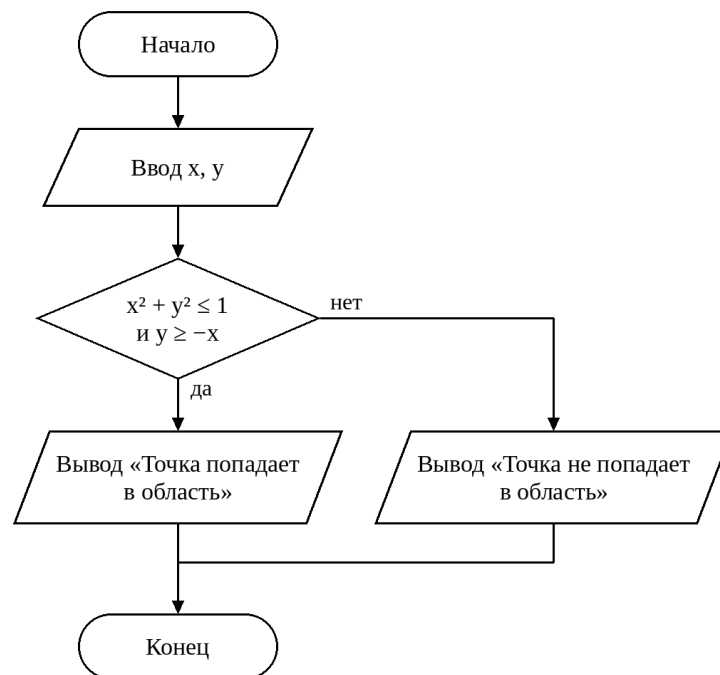


Рисунок 11 – Блок-схема алгоритма задачи № 3

Код программы:

```
1 #include <locale>
2 #include <iostream>
3
4 int main() {
5     setlocale(LC_ALL, "Russian");
6     double x, y;
7     std::cout << "Введите координаты точки (x y): ";
8     std::cin >> x >> y;
9
10    if (x * x + y * y <= 1 && y >= -x) {
11        std::cout << "Точка попадает в область\n";
12    } else {
13        std::cout << "Точка не попадает в область\n";
14    }
15    return 0;
16 }
```

Рисунок 12 – Код программы задачи № 3

Результат работы программы:

```
Введите координаты точки (x y): 0.3 0.4
Точка попадает в область
```

Рисунок 13 – Результат работы программы задачи № 3 (точка (0,3; 0,4))

```
Введите координаты точки (x y): -0.5 0.6
Точка попадает в область
```

Рисунок 14 – Результат работы программы задачи № 3 (точка (−0,5; 0,6))

```
Введите координаты точки (x y): 0.5 -0.7
Точка не попадает в область
```

Рисунок 15 – Результат работы программы задачи № 3 (точка (0,5; −0,7))

```
Введите координаты точки (x y): 0.9 0.9
Точка не попадает в область
```

Рисунок 16 – Результат работы программы задачи № 3 (точка (0,9; 0,9))

Проверка: для точки (0,3; 0,4) имеем $0,09 + 0,16 = 0,25 \leq 1$ и $0,4 \geq -0,3$ — точка внутри; для точки (−0,5; 0,6) имеем $0,25 + 0,36 = 0,61 \leq 1$ и $0,6 \geq 0,5$ — точка внутри; точка (0,5; −0,7) лежит ниже прямой $y = -x$ ($-0,7 < -0,5$); точка (0,9; 0,9) лежит вне круга ($0,81 + 0,81 = 1,62 > 1$).

№ 4

Ввести одномерный массив из n элементов. Вычислить сумму всех отрицательных чисел и сумму всех положительных чисел.

Ход решения:

- 1) Ввести с клавиатуры количество элементов n и выделить под массив динамическую память (операция new).
- 2) В цикле for ввести n элементов массива.
- 3) Присвоить суммам отрицательных (negSum) и положительных (posSum) чисел начальное значение 0.
- 4) Во втором цикле for перебрать все элементы: если $a[i] < 0$, прибавить элемент к negSum, иначе если $a[i] > 0$ — прибавить к posSum. Нулевые элементы не относятся ни к положительным, ни к отрицательным и пропускаются.
- 5) Вывести обе суммы на экран и освободить память (операция delete[]).

Блок-схема алгоритма:

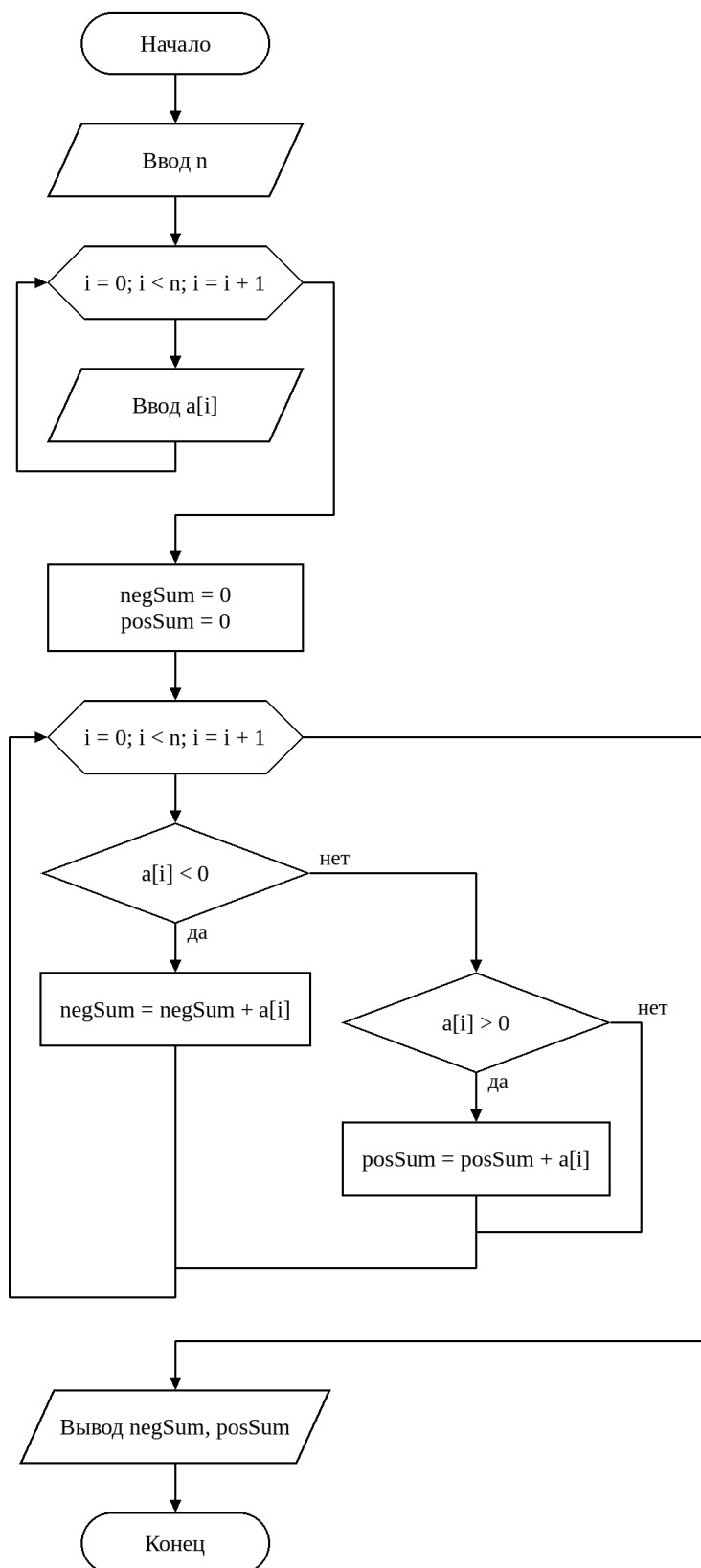


Рисунок 17 – Блок-схема алгоритма задачи № 4

Код программы:

```
1 #include <locale>
2 #include <iostream>
3
4 int main() {
5     setlocale(LC_ALL, "Russian");
6     int n;
7     std::cout << "Введите количество элементов n: ";
8     std::cin >> n;
9
10    double* a = new double[n];
11    std::cout << "Введите элементы массива: ";
12    for (int i = 0; i < n; i++) {
13        std::cin >> a[i];
14    }
15
16    double negSum = 0, posSum = 0;
17    for (int i = 0; i < n; i++) {
18        if (a[i] < 0) {
19            negSum = negSum + a[i];
20        } else if (a[i] > 0) {
21            posSum = posSum + a[i];
22        }
23    }
24
25    std::cout << "Сумма отрицательных чисел: " << negSum << "\n";
26    std::cout << "Сумма положительных чисел: " << posSum << "\n";
27    delete[] a;
28    return 0;
29 }
```

Рисунок 18 – Код программы задачи № 4

Результат работы программы:

```
Введите количество элементов n: 6
Введите элементы массива: 3 -5 7 0 -2 4.5
Сумма отрицательных чисел: -7
Сумма положительных чисел: 14.5
```

Рисунок 19 – Результат работы программы задачи № 4 (n = 6)

```
Введите количество элементов n: 4
Введите элементы массива: -1 -2.5 -3 -4
Сумма отрицательных чисел: -10.5
Сумма положительных чисел: 0
```

Рисунок 20 – Результат работы программы задачи № 4 (все элементы отрицательные)

Проверка: для массива 3, -5, 7, 0, -2, 4,5 сумма отрицательных чисел равна $-5 + (-2) = -7$, сумма положительных: $3 + 7 + 4,5 = 14,5$. Для массива из одних отрицательных чисел сумма положительных равна 0.

№ 5

Дано натуральное число N. Вычислить:

$$s = \sum_{i=1}^N \prod_{j=1}^i \frac{j!}{i!}$$

Ход решения:

- 1) Ввести с клавиатуры натуральное число N.
 - 2) Присвоить сумме s начальное значение 0, а переменной factI, в которой накапливается факториал i!, — значение 1.
 - 3) Во внешнем цикле for (i от 1 до N) домножить factI на i, получив i!. Произведению p и переменной factJ (для накопления j!) присвоить значение 1.
 - 4) Во внутреннем цикле for (j от 1 до i) домножить factJ на j, получив j!, и домножить произведение p на j! / i!.
 - 5) После завершения внутреннего цикла прибавить произведение p к сумме s.
 - 6) После завершения внешнего цикла вывести s на экран.
- Факториалы не вычисляются заново на каждой итерации, а накапливаются: $i! = (i - 1)! \cdot i$, $j! = (j - 1)! \cdot j$. Для переменных выбран тип double, так как факториалы быстро растут, а отношение $j! / i!$ является дробным.

Блок-схема алгоритма:

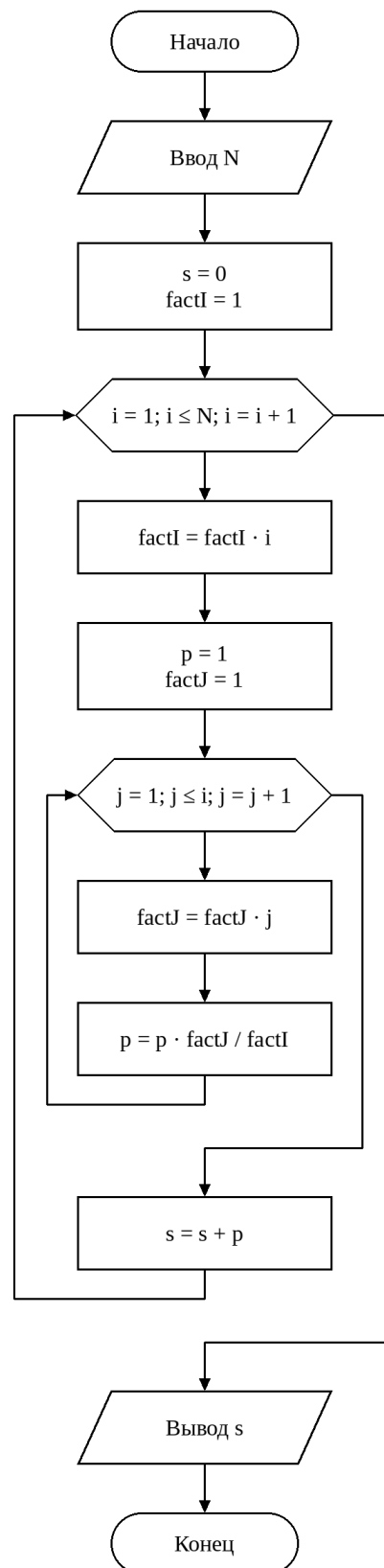


Рисунок 21 – Блок-схема алгоритма задачи № 5

Код программы:

```
1  #include <locale>
2  #include <iostream>
3
4  int main() {
5      setlocale(LC_ALL, "Russian");
6      int n;
7      std::cout << "Введите N: ";
8      std::cin >> n;
9
10     double s = 0;
11     double factI = 1;
12     for (int i = 1; i <= n; i++) {
13         factI = factI * i;
14         double p = 1;
15         double factJ = 1;
16         for (int j = 1; j <= i; j++) {
17             factJ = factJ * j;
18             p = p * factJ / factI;
19         }
20         s = s + p;
21     }
22
23     std::cout << "s = " << s << "\n";
24     return 0;
25 }
```

Рисунок 22 – Код программы задачи № 5

Результат работы программы:

```
Введите N: 1
s = 1
```

Рисунок 23 – Результат работы программы задачи № 5 (N = 1)

```
Введите N: 3
s = 1.55556
```

Рисунок 24 – Результат работы программы задачи № 5 (N = 3)

```
Введите N: 5
s = 1.55642
```

Рисунок 25 – Результат работы программы задачи № 5 (N = 5)

Проверка: при $N = 3$ получаем $s = 1 + (1! \cdot 2!) / (2!)^2 + (1! \cdot 2! \cdot 3!) / (3!)^3 = 1 + 2 / 4 + 12 / 216 = 1 + 0,5 + 0,055556 \approx 1,55556$, что совпадает с выводом программы.

Ответы на контрольные вопросы:

1. Опишите, из чего состоит схема программы.

Схема программы состоит из символов процесса, указывающих фактические операции обработки данных (включая символы, определяющие путь с учетом логических условий), линейных символов, указывающих поток управления, и специальных символов, облегчающих написание и чтение схемы (соединители, комментарии).

2. Дайте определение понятию «алгоритм».

Алгоритм — это конечная последовательность точно определенных действий (инструкций), выполнение которых приводит к решению поставленной задачи за конечное число шагов. Алгоритм обладает свойствами дискретности, определенности, результативности, конечности и массовости.

3. Укажите базовые структуры алгоритмов.

Базовыми структурами алгоритмов являются следование (линейная структура), ветвление (if, if...else, switch...case) и цикл (с предусловием, с постусловием и с параметром).

4. Раскройте понятие «условный оператор».

Условный оператор — оператор, который выбирает, какое действие выполнить, в зависимости от истинности логического условия. В C++ это оператор if (условие) { ... } else { ... }: если условие истинно, выполняется первая ветвь, иначе — ветвь else (она может отсутствовать).

5. Укажите, для чего используется блок процесса в блок-схемах.

Блок «Процесс» (прямоугольник) обозначает выполнение операции или группы операций, в результате которых изменяется значение, форма представления или расположение данных, например, присваивание или вычисление по формуле.

6. Укажите, для чего используется блок решения в блок-схемах.

Блок «Решение» (ромб) используется для выбора направления выполнения алгоритма в зависимости от условия. Он имеет один вход и не менее двух выходов; при двух выходах на них подписывается результат проверки «да» / «нет».

7. Укажите, какие параметры должны быть у блоков в блок-схемах алгоритмов.

Размеры символов определяются параметрами а и b: размер а выбирается из ряда 10, 15, 20 мм (допускается увеличивать на число, кратное 5 мм), размер b равен 1,5а. Записи внутри символа должны читаться слева направо и сверху вниз, основное направление потока — сверху вниз и слева направо, линии в неосновном направлении обозначаются стрелками.

8. Приведите примеры задачи и блок-схемы к ней, в которой используется оператор выбора.

Пример задачи: по номеру дня недели (от 1 до 7) вывести его название. В блок-схеме из блока «Решение», внутри которого записана переменная day, выходит несколько линий, подписанных значениями 1, 2, ..., 7; на каждой линии стоит блок вывода соответствующего названия, а отдельная линия «иначе» ведет к выводу сообщения об ошибке. Все линии затем сходятся перед блоком «Конец». В программе это реализуется оператором `switch (day) { case 1: ... default: ... }`.

9. Приведите примеры задачи и блок-схемы к ней, в которой используется циклический оператор.

Примером служит задача № 4 данного варианта (рисунок 17): для перебора элементов массива используется цикл `for`, изображенный блоком «Подготовка» (шестиугольник) с записью «`i = 0; i < n; i = i + 1`»; тело цикла выполняется для каждого значения `i`, после чего линия возвращается в блок цикла, а по окончании цикла управление передается следующему блоку. Вложенные циклы показаны в задаче № 5.

Вывод:

В ходе выполнения лабораторной работы были сформированы практические навыки по основам построения базовых алгоритмов. Были разработаны линейные (задачи № 1 и № 2), разветвляющийся (задача № 3) и циклические (задачи № 4 и № 5) алгоритмы, построены их блок-схемы в соответствии с ГОСТ 19.701-90, а сами алгоритмы реализованы на языке C++ с использованием арифметических операций, математических функций библиотеки `cmath`, условного оператора `if...else`, цикла `for`, в том числе вложенного, и динамического массива. Работа программ проверена на тестовых данных, результаты совпадают с ручным расчетом.